Weekly Mini Project 04 — Blood Donation Network

1. Purpose

This document explains how the **Blood Donation Network** project is organized.
The project is an emergency blood donation coordination platform built with **React + TypeScript** (using React Context for state) on the frontend, **NestJS + TypeScript** (with DTO validation, JWT authentication, role guards, Swagger, and custom domain logging) on the backend, and **PostgreSQL** accessed through **Prisma ORM** for persistent relational storage.
The backend decides every important business rule: blood-type red-cell compatibility (R-1, R-2), the 56-day post-donation rest period (R-3..R-5), donor eligibility and ranked matching (R-6..R-8), request lifecycle transitions (R-9..R-14), active pledge limits and automatic release (R-15..R-18), atomic donation recording inside a database transaction (R-19..R-22), and role/privacy enforcement (R-23, R-24). The frontend only displays what the backend returns and guides the user.
Every name, route, method signature, DTO property order, TypeScript interface, CSS variable, and file owner in this document is fixed. Need a new shared name or a contract change? Ask the team lead (Eyad Ibrahim) first.

2. Root Structure

```
blood-donation-network/
├── api/
│   ├── prisma/
│   │   ├── migrations/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── src/
│   ├── requests/
│   │   └── api.http
│   ├── .env.example
│   ├── package.json
│   └── nest-cli.json
├── client/
│   ├── public/
│   ├── src/
│   ├── .env.example
│   ├── package.json
│   └── vite.config.ts
├── docs/
│   ├── API.md
│   ├── PROJECT-STRUCTURE.md
│   └── TASKS.md
```

```
├── .github/
│   ├── CODEOWNERS
│   └── pull_request_template.md
├── .editorconfig
├── .prettierrc
├── .gitignore
└── README.md
```

Item	Value
Backend folder	api/
Frontend folder	client/
API prefix	/api
API URL (local)	http://localhost:3000/api
Swagger URL (local)	http://localhost:3000/api/docs
Client URL (local)	http://localhost:5173
Client env variable	VITE_API_URL
Backend env variables	DATABASE_URL, JWT_SECRET, JWT_EXPIRES_IN, PORT

3. Technology

Frontend

- React, TypeScript, Vite, React Router (react-router-dom)
- **React Context** for state (one context per feature: AuthContext, DonorContext, RequestsContext)
- Hooks: useState, useEffect, useCallback, useMemo, useContext
- Plain CSS, one file per page or component, using CSS variables from client/src/styles/theme.css and strict BEM owner prefixes
- API service layer in client/src/services/ (http.ts, auth.api.ts, donors.api.ts, pledges.api.ts, requests.api.ts)
- localStorage only for persisting the JWT token (bdn.token) managed exclusively by AuthContext / http.ts

Backend

- Node.js, NestJS, TypeScript
- **PostgreSQL** database accessed through **Prisma ORM** (prisma/schema.prisma, migrations, prisma/seed.ts)
- Architecture: **Controller → Service → Persistence (Prisma)**. Controllers contain zero business rules and zero database queries
- **JWT authentication** (@nestjs/jwt, bcrypt) with backend guards (JwtAuthGuard, RolesGuard) enforcing COORDINATOR and DONOR roles while keeping public endpoints (@Public()) deliberately open and returning public fields only
- **DTO classes** with class-validator and class-transformer
- One global ValidationPipe (common/validation.pipe.ts with whitelist: true, forbidNonWhitelisted: true, stopAtFirstError: true)

- One global AllExceptionsFilter (common/all-exceptions.filter.ts) returning { "message": "..." }
- **Swagger / OpenAPI** (@nestjs/swagger) at /api/docs documenting every endpoint, summary, resource tag, body, parameter, and Bearer auth requirement
- **Custom Logger** (common/app-logger.service.ts) recording meaningful domain events (request created, cancelled, fulfilled; donation recorded; rejected donation attempts with reason—never logging every HTTP request and never logging passwords or tokens)

Not allowed (Out of Scope)

- Redux, Zustand, React Query, or any external state management library
- Tailwind CSS, Bootstrap, Material UI, or any external UI/CSS component library
- Maps or external location/geocoding services
- SMS, email, or push notification services
- Payments or billing systems
- Real hospital/blood-bank ERP, laboratory/blood-testing systems, blood-unit inventory/storage/expiry tracking, or medical fitness diagnosis
- WebSockets, GraphQL, mobile apps, Docker, or cloud deployment

Only the team lead (Eyad Ibrahim) installs npm packages. Need a package? Ask first. This prevents package-lock.json conflicts.

4. Prisma Schema Design (api/prisma/schema.prisma)

Owner: **Eyad Ibrahim**.
All data is stored in PostgreSQL and survives a backend restart. We store unitsCollected directly on BloodRequest and update it atomically inside prisma.$transaction when recording a donation (R-22), while unitsStillNeeded (unitsRequired - unitsCollected, R-10), donor eligibility status (ELIGIBLE / REST_PERIOD / UNAVAILABLE / INACTIVE), and nextAllowedDonationDate (lastDonationDate + 56 days, R-4) are calculated dynamically by the backend on every read.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  COORDINATOR
  DONOR
}

enum BloodType {
  O_NEG
  O_POS
```

```
  A_NEG
  A_POS
  B_NEG
  B_POS
  AB_NEG
  AB_POS
}

enum Urgency {
  CRITICAL
  HIGH
  NORMAL
}

enum RequestStatus {
  OPEN
  FULFILLED
  CANCELLED
}

enum PledgeStatus {
  PLEDGED
  COMPLETED
  WITHDRAWN
  RELEASED
}

model Area {
  id        String         @id @default(uuid())
  name      String         @unique
  donors    DonorProfile[]
  requests  BloodRequest[]
  createdAt DateTime       @default(now())
}

model User {
  id                String         @id @default(uuid())
  email             String?        @unique
  phone             String         @unique
  passwordHash      String
  role              Role
  isActive          Boolean        @default(true)
  createdAt         DateTime       @default(now())
  updatedAt         DateTime       @updatedAt
  donorProfile      DonorProfile?
  createdRequests   BloodRequest[] @relation("CoordinatorRequests")
  recordedDonations Donation[]     @relation("CoordinatorDonations")
}
```

```
model DonorProfile {
  id               String     @id @default(uuid())
  userId           String     @unique
  user             User       @relation(fields: [userId], references:
[id], onDelete: Cascade)
  fullName         String
  bloodType        BloodType
  areaId           String
  area             Area       @relation(fields: [areaId], references:
[id])
  isAvailable      Boolean    @default(true)
  lastDonationDate DateTime?
  createdAt        DateTime   @default(now())
  updatedAt        DateTime   @updatedAt
  pledges          Pledge[]
  donations        Donation[]
}

model BloodRequest {
  id               String        @id @default(uuid())
  patientReference String
  bloodType        BloodType
  unitsRequired    Int
  unitsCollected   Int           @default(0)
  areaId           String
  area             Area          @relation(fields: [areaId],
references: [id])
  urgency          Urgency
  status           RequestStatus @default(OPEN)
  hospital         String
  note             String?
  cancelReason     String?
  createdById      String
  createdBy        User          @relation("CoordinatorRequests",
fields: [createdById], references: [id])
  createdAt        DateTime      @default(now())
  updatedAt        DateTime      @updatedAt
  pledges          Pledge[]
  donations        Donation[]

  @@index([status, urgency, createdAt])
  @@index([patientReference, bloodType, status])
}

model Pledge {
  id        String       @id @default(uuid())
  donorId   String
```

```
  donor     DonorProfile @relation(fields: [donorId], references:
[id])
  requestId String
  request   BloodRequest @relation(fields: [requestId], references:
[id])
  status    PledgeStatus @default(PLEDGED)
  createdAt DateTime     @default(now())
  updatedAt DateTime     @updatedAt

  @@index([donorId, status])
  @@index([requestId, status])
}

model Donation {
  id           String       @id @default(uuid())
  donorId      String
  donor        DonorProfile @relation(fields: [donorId], references:
[id])
  requestId    String
  request      BloodRequest @relation(fields: [requestId], references:
[id])
  recordedById String
  recordedBy   User         @relation("CoordinatorDonations", fields:
[recordedById], references: [id])
  donationDate DateTime
  units        Int          @default(1)
  createdAt    DateTime     @default(now())

  @@index([requestId])
  @@index([donorId, donationDate])
}
```

BloodType Serialization Rule

In the database Prisma schema, enum keys are O_NEG, O_POS, A_NEG, A_POS, B_NEG, B_POS, AB_NEG, AB_POS. Across the **REST API JSON contract and Frontend**, blood types are always represented in standard clinical notation: 'O-' | 'O+' | 'A-' | 'A+' | 'B-' | 'B+' | 'AB-' | 'AB+' (using normal ASCII hyphen - and plus +). normalize.util.ts provides toApiBloodType(prismaEnum) and toPrismaBloodType(apiString) so all API payloads and responses use 'O-', 'A-', etc.

5. Frontend Structure

```
client/src/
├── components/
|   ├── common/
```

```
│   │   ├── Loading.tsx
│   │   ├── Loading.css
│   │   ├── ErrorMessage.tsx
│   │   ├── ErrorMessage.css
│   │   ├── EmptyState.tsx
│   │   ├── EmptyState.css
│   │   ├── Button.tsx
│   │   ├── Button.css
│   │   ├── StatusBadge.tsx
│   │   ├── StatusBadge.css
│   │   ├── UrgencyBadge.tsx
│   │   ├── UrgencyBadge.css
│   │   ├── BloodTypeBadge.tsx
│   │   ├── BloodTypeBadge.css
│   │   ├── PageContainer.tsx
│   │   └── PageContainer.css
│   └── layout/
│       ├── Layout.tsx
│       ├── Layout.css
│       ├── Navbar.tsx
│       ├── Navbar.css
│       ├── ProtectedRoute.tsx
│       └── ProtectedRoute.css
├── context/
│   ├── AuthContext.tsx
│   ├── DonorContext.tsx
│   └── RequestsContext.tsx
├── pages/
│   ├── SosBoard/
│   │   ├── SosBoardPage.tsx
│   │   ├── SosBoardPage.css
│   │   └── SosRequestCard.tsx
│   ├── SignIn/
│   │   ├── SignInPage.tsx
│   │   └── SignInPage.css
│   ├── SignUp/
│   │   ├── SignUpPage.tsx
│   │   └── SignUpPage.css
│   ├── DonorDashboard/
│   │   ├── DonorDashboardPage.tsx
│   │   ├── DonorDashboardPage.css
│   │   ├── EligibilityBanner.tsx
│   │   ├── ActivePledgeCard.tsx
│   │   ├── EligibleRequestsList.tsx
│   │   └── DonationHistoryTable.tsx
│   ├── DonorProfile/
│   │   ├── DonorProfilePage.tsx
│   │   └── DonorProfilePage.css
```

```
|   ├── Requests/
|   |   ├── RequestsPage.tsx
|   |   ├── RequestsPage.css
|   |   ├── RequestFiltersBar.tsx
|   |   └── CreateRequestModal.tsx
|   ├── RequestDetails/
|   |   ├── RequestDetailsPage.tsx
|   |   ├── RequestDetailsPage.css
|   |   ├── UpdateRequestForm.tsx
|   |   ├── CancelRequestModal.tsx
|   |   └── RecordDonationModal.tsx
|   ├── EligibleDonors/
|   |   ├── EligibleDonorsPage.tsx
|   |   ├── EligibleDonorsPage.css
|   |   ├── ExcludedDonorsSummary.tsx
|   |   └── EligibleDonorRow.tsx
|   └── NotFound/
|       ├── NotFoundPage.tsx
|       └── NotFoundPage.css
├── routes/
|   └── AppRouter.tsx
├── services/
|   ├── http.ts
|   ├── auth.api.ts
|   ├── donors.api.ts
|   ├── pledges.api.ts
|   └── requests.api.ts
├── types/
|   ├── auth.types.ts
|   ├── donor.types.ts
|   └── request.types.ts
├── utils/
|   ├── format.ts
|   └── date.ts
├── styles/
|   ├── theme.css
|   └── global.css
├── constants.ts
├── App.tsx
└── main.tsx
```

Every component has its own .css file. Page owners may add small sub-components inside their own page folder.

5.1 Routes

All routes are created in Phase 0 by Eyad Ibrahim and point to placeholder pages. Only the team lead edits AppRouter.tsx.

Route	Page	Access / Role	Owner
/	SosBoardPage	Everyone (Public)	Molham
/sign-in	SignInPage	Everyone (Public)	Molham
/sign-up	SignUpPage	Everyone (Public — Donor registration)	Molham
/donor/dashboard	DonorDashboardPage	DONOR	Taymaa
/donor/profile	DonorProfilePage	DONOR	Taymaa
/coordinator/requests	RequestsPage	COORDINATOR	Rami
/coordinator/requests/:requestId	RequestDetailsPage	COORDINATOR	Rami
/coordinator/requests/:requestId/eligible-donors	EligibleDonorsPage	COORDINATOR	Rami
*	NotFoundPage	Everyone	Molham

Navigation & Role-Based Sign-In Prompt Rule (Section 10)

- **When not signed in**, the Navbar shows: App Logo/Name, **SOS Board** (/), **Sign In** (/sign-in), and **Become a Donor** (/sign-up).
- **When signed in as DONOR**, the Navbar shows: **SOS Board** (/), **My Dashboard** (/donor/dashboard), **My Profile** (/donor/profile), the donor's name + blood type badge, and **Sign Out**.
- **When signed in as COORDINATOR**, the Navbar shows: **SOS Board** (/), **Manage Requests** (/coordinator/requests), the coordinator's phone/email + Coordinator badge, and **Sign Out**.
- **Protected Screens (ProtectedRoute.tsx)**: Per Section 10 ("When there is no valid sign-in, screens that need one show a sign-in prompt instead of their content"), when an unauthenticated visitor opens /donor/* or /coordinator/*, ProtectedRoute renders a clear **Sign-In Required** prompt card with a button linking to /sign-in (instead of rendering page content or a blank screen). If a signed-in user has the wrong role (e.g., a DONOR opens /coordinator/requests), ProtectedRoute shows a clear **Access Restricted** message with a link back to their own dashboard.

5.2 State with Context

Server data (with its loading and error state) lives in **three contexts**, one for each feature domain:

```
client/src/context/
├── AuthContext.tsx        owner: Molham
├── DonorContext.tsx       owner: Taymaa
└── RequestsContext.tsx    owner: Rami
```

Each file exports a **Provider** and a **hook**:

```
AuthProvider        useAuth()
DonorProvider       useDonor()
RequestsProvider    useRequests()
```

App.tsx wraps AppRouter with all three providers (AuthProvider outermost, then DonorProvider

and RequestsProvider).

The pattern (all three contexts use it)

```
const RequestsContext = createContext<RequestsContextValue |
undefined>(undefined);

export function RequestsProvider({ children }: { children: ReactNode
}) {
  const [sosRequests, setSosRequests] =
useState<PublicSosRequest[]>([]);
  const [isSosLoading, setIsSosLoading] = useState(false);
  const [sosError, setSosError] = useState<string | null>(null);

  const loadSosRequests = useCallback(async () => {
    setIsSosLoading(true);
    setSosError(null);
    try {
      setSosRequests(await getSosRequests());
    } catch (err) {
      setSosError(err instanceof Error ? err.message : 'Something went
wrong. Please try again.');
    } finally {
      setIsSosLoading(false);
    }
  }, []);

  const value = useMemo(
    () => ({ sosRequests, isSosLoading, sosError, loadSosRequests /*
...other fields */ }),
    [sosRequests, isSosLoading, sosError, loadSosRequests],
  );

  return <RequestsContext.Provider
value={value}>{children}</RequestsContext.Provider>;
}

export function useRequests() {
  const context = useContext(RequestsContext);
  if (!context) throw new Error('useRequests must be used inside
RequestsProvider');
  return context;
}
```

Rules

- Every action that can fail catches the error and saves a human-readable message from

ApiError.message. A page is never left loading forever.
- Pages and components **never** call services/ or fetch directly. Only contexts do.
- A page reloads its data when it mounts (useEffect), and mutating actions re-fetch or update affected state immediately so counters and lists update without a manual page reload.
- A context only holds what the API returned. It never computes blood compatibility, rest-period dates, or eligibility on the client.

Fixed context shapes

AuthContext (Molham)

```
user: AuthUser | null;
token: string | null;
areas: Area[];
isAuthLoading: boolean;          // true during initial /auth/me check
or sign-in/sign-up submission
isAreasLoading: boolean;
authError: string | null;

loadAreas(): Promise<void>;
loadCurrentUser(): Promise<void>;
signIn(payload: SignInPayload): Promise<AuthUser | null>;     // null
if failed
signUp(payload: SignUpPayload): Promise<AuthUser | null>;     // null
if failed
signOut(): void;
clearAuthError(): void;
```

DonorContext (Taymaa)

```
profile: DonorDashboardProfile | null;
isProfileLoading: boolean;
profileError: string | null;
loadMyProfile(): Promise<void>;

eligibleRequests: EligibleOpenRequest[];
isEligibleRequestsLoading: boolean;
eligibleRequestsError: string | null;
loadEligibleRequests(): Promise<void>;

isActionLoading: boolean;        // true while pledge / withdraw /
profile update / deactivate runs
actionError: string | null;

updateMyProfile(payload: UpdateDonorProfilePayload): Promise<boolean>;
deactivateMyAccount(): Promise<boolean>;
createPledge(requestId: string): Promise<boolean>;
withdrawPledge(pledgeId: string): Promise<boolean>;
clearDonorErrors(): void;
```

When createPledge or withdrawPledge succeeds, DonorContext immediately calls loadMyProfile() and loadEligibleRequests() so the active pledge card, buttons, and counters update without a page reload.

RequestsContext (Rami)

```
sosRequests: PublicSosRequest[];
isSosLoading: boolean;
sosError: string | null;
loadSosRequests(): Promise<void>;

requests: CoordinatorRequestSummary[];
isRequestsLoading: boolean;
requestsError: string | null;
loadRequests(filters?: RequestFilters): Promise<void>;

currentRequest: CoordinatorRequestDetails | null;
isRequestDetailsLoading: boolean;
requestDetailsError: string | null;
loadRequestById(requestId: string): Promise<void>;

eligibleDonorsResult: EligibleDonorsResponse | null;
isEligibleDonorsLoading: boolean;
eligibleDonorsError: string | null;
loadEligibleDonors(requestId: string): Promise<void>;

allDonors: CoordinatorDonorItem[];
isAllDonorsLoading: boolean;
allDonorsError: string | null;
loadAllDonors(): Promise<void>;

isMutationLoading: boolean;     // true while create / update / cancel
/ recordDonation runs
mutationError: string | null;

createRequest(payload: CreateRequestPayload):
Promise<CoordinatorRequestDetails | null>;
updateRequest(requestId: string, payload: UpdateRequestPayload):
Promise<boolean>;
cancelRequest(requestId: string, payload: CancelRequestPayload):
Promise<boolean>;
recordDonation(requestId: string, payload: RecordDonationPayload):
Promise<boolean>;
clearMutationError(): void;
```

When updateRequest, cancelRequest, or recordDonation succeeds, RequestsContext immediately reloads loadRequestById(requestId) (and loadEligibleDonors(requestId) if on the Eligible Donors screen) so units collected, units still needed, request status, pledges, and

donations update without a manual page reload.

6. Pages

SOS Board (/) — Owner: Molham

Uses useRequests() (sosRequests, isSosLoading, sosError, loadSosRequests) and useAuth().

- Displays a crimson hero banner explaining the platform's mission ("When a patient needs blood urgently, find the right donors in the right area — fast") with call-to-action buttons (**Become a Donor** / **Sign In**, or **Go to Dashboard** if signed in).
- Displays all OPEN requests ordered by urgency (CRITICAL → HIGH → NORMAL) and then by creation time (oldest first), exactly as returned by GET /api/requests/sos.
- Each SosRequestCard shows **only public data** (FR-8, R-23): BloodTypeBadge, UrgencyBadge, area name, hospital name, unitsStillNeeded ("X units still needed"), and relative/formatted creation time. Never shows patient references, notes, or donor personal data.
- Includes Loading, ErrorMessage (with retry), and EmptyState ("No active SOS blood requests right now. All current hospital needs have been met.") when there are zero open requests (Section 13: "Empty state, not an error").

Sign In (/sign-in) & Donor Sign Up (/sign-up) — Owner: Molham

Uses useAuth().

- **Sign In (/sign-in)**: Form with identifier (phone number or email) and password. Supports signing in as either a COORDINATOR or a DONOR (FR-2). Shows authError inline next to the form (never only in console). On success, redirects COORDINATOR to /coordinator/requests and DONOR to /donor/dashboard.
- **Donor Sign Up (/sign-up)**: Calls loadAreas() on mount. Form with fullName, phone, optional email, password, bloodType selector (all 8 blood types), and areaId selector (populated from GET /api/areas) (FR-1). Shows validation and conflict errors (400 / 409) inline inside the form. On success, signs the donor in and redirects to /donor/dashboard.

Donor Dashboard (/donor/dashboard) — Owner: Taymaa

Uses useDonor() (profile, eligibleRequests, loadMyProfile, loadEligibleRequests, createPledge, withdrawPledge).

- **Eligibility Status Banner (EligibilityBanner)**: Displays the donor's calculated eligibilityStatus from the backend (FR-4):
  - ELIGIBLE: Green status banner — "Eligible to donate now".
  - REST_PERIOD: Amber status banner — "In rest period until [formatted nextAllowedDonationDate]" and explains the 56-day post-donation cooldown.
  - UNAVAILABLE: Neutral/muted banner — "Currently marked as unavailable" with a quick link to /donor/profile to turn availability back on.
  - INACTIVE: Red banner — "Account is deactivated".
- **Active Pledge Card (ActivePledgeCard)**: Shows the donor's current PLEDGED pledge (at most one across all requests, R-16) with blood type, hospital, area, urgency badge, and a **Withdraw pledge** button (PATCH /api/pledges/:id/withdraw). When clicked,

updates the active pledge card and eligible requests list immediately without reload.
- **Requests I Can Help With (EligibleRequestsList)**: Displays GET /api/donors/me/eligible-requests (FR-10) ordered like the SOS board (CRITICAL first, then oldest first). Each item has a **Pledge to donate** button (POST /api/requests/:id/pledges). Any business-rule rejection (e.g., already holding an active pledge, rest period, or active pledges reached unitsStillNeeded) is shown clearly inline (actionError). Has loading, error, and empty states.
- **Donation History (DonationHistoryTable)**: Lists the donor's past recorded donations (date, hospital, area, blood type, 1 unit) (FR-4), with an empty state if the donor has not donated yet.

Donor Profile (/donor/profile) — Owner: Taymaa

Uses useDonor() and useAuth() (for areas and signOut).
- Displays read-only identity fields (fullName, bloodType, email, lastDonationDate, nextAllowedDonationDate).
- **Edit Profile Form**: Allows the donor to update their areaId, phone, and isAvailable toggle (Available / Unavailable) via PATCH /api/donors/me (FR-3, R-24). Shows a success confirmation banner and inline backend validation/conflict errors.
- **Deactivate Account Section**: Danger zone with a confirmation step calling PATCH /api/donors/me/deactivate (FR-3). Deactivating sets isActive = false, automatically withdraws any active pledge, and signs the user out (or updates state).

Coordinator Requests (/coordinator/requests) — Owner: Rami

Uses useRequests() (requests, loadRequests, createRequest) and useAuth() (areas, loadAreas).
- **Filter Bar (RequestFiltersBar)**: Filter controls for status (All, OPEN, FULFILLED, CANCELLED), bloodType (All or any of the 8 types), areaId (All or any area), and urgency (All, CRITICAL, HIGH, NORMAL) (FR-7). Changing filters updates URL query parameters and calls loadRequests(filters).
- **Create SOS Request Modal/Form (CreateRequestModal)**: Allows the coordinator to publish a new SOS request with patientReference (internal code, not a patient name), bloodType, unitsRequired (integer 1–10), areaId, urgency, hospital, and optional note (FR-5). Displays backend 400 or 409 (duplicate open request with same patientReference and bloodType, R-13) errors clearly next to the form.
- **Requests Table / Grid**: Shows each request's patient reference, blood type badge, urgency badge, status badge (OPEN, FULFILLED, CANCELLED), area, hospital, progress (unitsCollected / unitsRequired and unitsStillNeeded), creation time, and links to **Request Details** (/coordinator/requests/:requestId) and **Eligible Donors** (/coordinator/requests/:requestId/eligible-donors).

Request Details (/coordinator/requests/:requestId) — Owner: Rami

Uses useRequests() (currentRequest, loadRequestById, updateRequest, cancelRequest, recordDonation).
- Shows all request data: patientReference, bloodType, urgency, status, area, hospital, note, cancelReason (if cancelled), createdAt, and unit counters: **Units Required**, **Units**

Collected, and **Units Still Needed** (FR-13).
- Link button to **Find Eligible Donors** (/coordinator/requests/:requestId/eligible-donors) when status === 'OPEN'.
- **Coordinator Actions (only enabled when status === 'OPEN', R-9, R-14)**:
  - **Update Request (UpdateRequestForm)**: Edit unitsRequired (cannot be less than unitsCollected, R-12), urgency, and note (PATCH /api/requests/:id).
  - **Cancel Request (CancelRequestModal)**: Cancel with a required cancelReason (PATCH /api/requests/:id/cancel, Flow E).
  - **Record Donation (RecordDonationModal)**: Record a completed donation for a donor (donorId and donationDate, defaulting to today, POST /api/requests/:id/donations, Flow D).
- **Pledges List**: Shows all pledges for this request (donor full name, blood type, phone, status badge PLEDGED / COMPLETED / WITHDRAWN / RELEASED, pledge time) and a quick **Record Donation** button next to any PLEDGED donor.
- **Recorded Donations List**: Shows all completed donations on this request (donor name, blood type, donation date, 1 unit, recorded by coordinator).
- All backend rejections (400 / 409, such as trying to record a donation for a donor in rest period with exact date) are displayed prominently on screen. After any action, counters, status, pledges, and donations update immediately without a page reload.

Eligible Donors (/coordinator/requests/:requestId/eligible-donors) — Owner: Rami

Uses useRequests() (currentRequest, eligibleDonorsResult, loadRequestById, loadEligibleDonors, recordDonation).
- Displays a summary header of the target request (patient reference, needed blood type, area, urgency, units still needed) and a link back to /coordinator/requests/:requestId.
- **Excluded Donors Summary (ExcludedDonorsSummary)**: Displays the backend's excludedSummary counts of compatible donors in the same area who were excluded, grouped by reason (FR-9, R-8):
  - **In Rest Period (restPeriod)**: count of compatible donors in the area currently inside their 56-day cooldown.
  - **Unavailable (unavailable)**: count of active, non-resting compatible donors in the area who set isAvailable = false.
  - **Inactive (inactive)**: count of compatible donor accounts in the area with isActive = false.
- **Ranked Eligible Donors List (EligibleDonorRow)**: Displays the ranked list returned by GET /api/requests/:id/eligible-donors (FR-9, R-7: exact blood-type match first, then other compatible types; within each group, longest time since last donation or never donated first; then alphabetical by name). Each row shows rank number, donor full name, phone, BloodTypeBadge, an **Exact Match** vs **Compatible** tag, area name, lastDonationDate (or "Never donated"), whether they have an active pledge on this request, and a **Record Donation** button that opens the record-donation confirmation right away.

Not Found (*) — Owner: Molham

A friendly 404 page with a link back to the Public SOS Board (/).

7. Components

Shared components (client/src/components/common/)

Props are fixed:

Component	Props	Owner
Loading	message?: string	Molham
ErrorMessage	message: string, onRetry?: () => void	Molham
EmptyState	title: string, description?: string, action?: ReactNode	Molham
Button	variant?: 'primary' | 'secondary' | 'danger' | 'outline', size?: 'sm' | 'md' | 'lg', isLoading?: boolean, plus standard <button> props	Molham
StatusBadge	status: RequestStatus | PledgeStatus | EligibilityStatus	Molham
UrgencyBadge	urgency: Urgency	Molham
BloodTypeBadge	bloodType: BloodType, isExactMatch?: boolean	Molham
PageContainer	title?: string, subtitle?: string, actions?: ReactNode, children: ReactNode	Eyad

Badge visual rules:

- UrgencyBadge: CRITICAL uses --color-critical (crimson red), HIGH uses --color-warning (amber/orange), NORMAL uses --color-info (blue).
- StatusBadge:
  - Request statuses: OPEN (crimson/active), FULFILLED (success green), CANCELLED (neutral gray).
  - Pledge statuses: PLEDGED (info blue), COMPLETED (success green), WITHDRAWN (neutral gray), RELEASED (amber/muted).
  - Donor eligibility statuses: ELIGIBLE (success green), REST_PERIOD (warning amber), UNAVAILABLE (neutral gray), INACTIVE (error red).
- Color is never the only signal; every badge renders an explicit text label.

In Phase 0, the team lead (Eyad) creates working placeholder versions of these shared components with these exact props so every page can import them from Day 1.

8. Backend Structure

```
api/
├── prisma/
│   ├── migrations/
│   ├── schema.prisma
│   └── seed.ts
```

```
├── src/
│   ├── common/
│   │   ├── constants.ts
│   │   ├── clock.service.ts
│   │   ├── app-logger.service.ts
│   │   ├── common.module.ts
│   │   ├── normalize.util.ts
│   │   ├── validators.ts
│   │   ├── validation.pipe.ts
│   │   └── all-exceptions.filter.ts
│   ├── prisma/
│   │   ├── prisma.module.ts
│   │   └── prisma.service.ts
│   ├── auth/
│   │   ├── decorators/
│   │   │   ├── public.decorator.ts
│   │   │   ├── roles.decorator.ts
│   │   │   └── current-user.decorator.ts
│   │   ├── guards/
│   │   │   ├── jwt-auth.guard.ts
│   │   │   └── roles.guard.ts
│   │   ├── dto/
│   │   │   ├── sign-up.dto.ts
│   │   │   └── sign-in.dto.ts
│   │   ├── auth.module.ts
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   └── auth.types.ts
│   ├── areas/
│   │   ├── areas.module.ts
│   │   ├── areas.controller.ts
│   │   └── areas.service.ts
│   ├── matching/
│   │   ├── matching.module.ts
│   │   ├── matching.service.ts
│   │   └── matching.types.ts
│   ├── requests/
│   │   ├── dto/
│   │   │   ├── create-request.dto.ts
│   │   │   ├── update-request.dto.ts
│   │   │   ├── cancel-request.dto.ts
│   │   │   ├── request-filters-query.dto.ts
│   │   │   └── request-id-param.dto.ts
│   │   ├── requests.module.ts
│   │   ├── requests.controller.ts
│   │   ├── requests.service.ts
│   │   └── requests.types.ts
│   ├── donations/
```

```
│   │   ├── dto/
│   │   │   └── record-donation.dto.ts
│   │   ├── donations.module.ts
│   │   ├── donations.controller.ts
│   │   └── donations.service.ts
│   ├── donors/
│   │   ├── dto/
│   │   │   └── update-donor-profile.dto.ts
│   │   ├── donors.module.ts
│   │   ├── donors.controller.ts
│   │   ├── donors.service.ts
│   │   └── donors.types.ts
│   ├── pledges/
│   │   ├── dto/
│   │   │   └── pledge-id-param.dto.ts
│   │   ├── pledges.module.ts
│   │   ├── pledges.controller.ts
│   │   └── pledges.service.ts
│   ├── app.module.ts
│   └── main.ts
└── requests/
    └── api.http
```

8.1 common/ (Owner: Eyad Ibrahim)

File	What it does	Owner
constants.ts	Single source of truth for REST_PERIOD_DAYS = 56, MIN_UNITS = 1, MAX_UNITS = 10, BLOOD_COMPATIBILITY_MAP, and URGENCY_PRIORITY	Eyad
clock.service.ts	ClockService.now(), the only source of the current date/time in services	Eyad
app-logger.service.ts	AppLogger recording domain events (logRequestCreated, logRequestCancelled, logRequestFulfilled, logDonationRecorded, warnDonationRejected) — never logs passwords or tokens	Eyad
common.module.ts	@Global() module exporting ClockService and AppLogger	Eyad
normalize.util.ts	normalizePhone, normalizeEmail,	Eyad

File	What it does	Owner
	normalizePatientRef, toApiBloodType, toPrismaBloodType, toDateOnlyString	
validators.ts	Custom class-validator decorators (@IsValidBloodType(), @IsCalendarDate(), @IsValidPhone())	Eyad
validation.pipe.ts	createValidationPipe() (whitelist: true, forbidNonWhitelisted: true, stopAtFirstError: true, single string { "message": "..." })	Eyad
all-exceptions.filter.ts	Global exception filter turning every HTTP or unexpected error into { "message": "..." }	Eyad

Business constants live **only** in api/src/common/constants.ts (enforcing R-1, R-2, R-3):

```
export const REST_PERIOD_DAYS = 56;
export const MIN_UNITS = 1;
export const MAX_UNITS = 10;

export const API_BLOOD_TYPES = [
  'O-',
  'O+',
  'A-',
  'A+',
  'B-',
  'B+',
  'AB-',
  'AB+',
] as const;

export type ApiBloodType = (typeof API_BLOOD_TYPES)[number];

/**
 * R-1: Standard red-cell compatibility table.
 * Key = Blood type needed by the request.
 * Value = Compatible donor blood types (exact match listed first).
 */
export const BLOOD_COMPATIBILITY_MAP: Record<ApiBloodType, readonly
ApiBloodType[]> = {
  'O-': ['O-'],
  'O+': ['O+', 'O-'],
  'A-': ['A-', 'O-'],
  'A+': ['A+', 'A-', 'O+', 'O-'],
```

```
  'B-': ['B-', 'O-'],
  'B+': ['B+', 'B-', 'O+', 'O-'],
  'AB-': ['AB-', 'A-', 'B-', 'O-'],
  'AB+': ['AB+', 'AB-', 'A+', 'A-', 'B+', 'B-', 'O+', 'O-'],
};

export const URGENCY_SORT_ORDER: Record<'CRITICAL' | 'HIGH' |
'NORMAL', number> = {
  CRITICAL: 1,
  HIGH: 2,
  NORMAL: 3,
};
```

8.2 Database & Seed (prisma/seed.ts — Owner: Eyad Ibrahim)

Per Section 12 ("Persistence: all data survives a restart; a seed script creates the areas, at least 2 coordinators, at least 20 donors covering all eight blood types (some in rest period, some unavailable), and several requests in different states"):

- **Supported Areas (6 areas)**: Homs, Damascus, Aleppo, Latakia, Hama, Tartus.
- **Coordinators (2 accounts)**:
  - Coordinator 1: coordinator.homs@bloodnet.sy / 0911000001 (password: Coordinator123!)
  - Coordinator 2: coordinator.damascus@bloodnet.sy / 0911000002 (password: Coordinator123!)
- **Donors (24 accounts covering all 8 blood types)**:
  - Includes the exact scenario actors from Section 6:
    - **Sara** (A-, area Homs, active, available, no recent donation — immediately eligible for the Homs A- request)
    - **Rami** (O-, area Homs, active, available, lastDonationDate = 20 days ago — excluded due to rest period)
  - Covers all 8 blood types (O-, O+, A-, A+, B-, B+, AB-, AB+) across areas, including donors with lastDonationDate = null (never donated), donors with lastDonationDate > 56 days ago (eligible), donors in rest period (< 56 days ago), unavailable donors (isAvailable = false), and an inactive donor (isActive = false) so the excluded-donor summary (restPeriod, unavailable, inactive) can be demonstrated immediately.
- **Blood Requests (6 seeded requests)**:
  - Multiple OPEN requests across CRITICAL, HIGH, and NORMAL urgencies (including the Section 6 A-, 3 units, Homs, CRITICAL request at Central Blood Bank Homs), plus FULFILLED and CANCELLED requests, existing PLEDGED / COMPLETED / RELEASED pledges, and historical Donation records.

8.3 Central Matching & Eligibility Engine (api/src/matching/ — Owner: Eyad Ibrahim)

MatchingService centralizes all blood compatibility, rest-period, and donor-ranking rules so neither RequestsService, DonationsService, DonorsService, nor PledgesService ever

duplicates rule logic:

Method	Responsibility & Rules Enforced
isBloodCompatible(neededType, donorType)	Returns true if donorType is in BLOOD_COMPATIBILITY_MAP[neededType] (R-1, R-2)
getCompatibleDonorTypes(neededType)	Returns the compatible donor blood types for neededType (R-1)
getNextAllowedDonationDate(lastDonationDate)	Returns null if lastDonationDate is null (R-5); otherwise returns lastDonationDate + REST_PERIOD_DAYS (56 days) as YYYY-MM-DD (R-3, R-4)
isInRestPeriod(lastDonationDate, now)	Returns true when lastDonationDate exists and today < nextAllowedDonationDate (R-4, R-5)
getDonorEligibilityStatus(donor, now)	Returns 'INACTIVE' if !donor.user.isActive, 'REST_PERIOD' if in rest period, 'UNAVAILABLE' if !donor.isAvailable, otherwise 'ELIGIBLE' (FR-4)
checkDonorEligibilityForRequest(donor, request, now)	Checks R-6 in exact order: (1) account active, (2) donor available, (3) not in rest period (returning nextAllowedDonationDate in message), (4) blood type compatible, (5) area matches request area. Returns { eligible: boolean; reason?: string; code?: 'INACTIVE' | 'UNAVAILABLE' | 'REST_PERIOD' | 'INCOMPATIBLE_BLOOD' | 'DIFFERENT_AREA' }
getEligibleDonorsForRequest(requestId)	Evaluates all donors in the request's area with compatible blood types. Separates excluded donors into { restPeriod, unavailable, inactive } counts (R-8) and orders eligible donors by: (1) exact blood-type match (donor.bloodType === request.bloodType) before other compatible types, (2) oldest lastDonationDate first (null / never donated comes first), (3) fullName ascending (localeCompare) (R-7)

8.4 Validation and DTOs

Every backend member writes the DTOs for **their own module**. Validation has two layers:

Layer	Where	Examples
Format	DTO classes (class-validator)	non-empty string, valid blood type (O-..AB+), integer unitsRequired between 1 and 10, valid phone format, real calendar date YYYY-MM-DD, unknown fields rejected (forbidNonWhitelisted: true)
Business Rules	Services (MatchingService,	duplicate phone/email (409),

Layer	Where	Examples
	RequestsService, DonationsService, DonorsService, PledgesService)	area exists (400/404), duplicate OPEN request for same patientReference + bloodType (409), unitsRequired >= unitsCollected (400), request must be OPEN (409), rest period / compatibility / area / active pledge checks, donation date not in future (400)

Custom decorators (common/validators.ts — Owner: Eyad)

Decorator	Passes when
@IsValidBloodType()	String is one of 'O-', 'O+', 'A-', 'A+', 'B-', 'B+', 'AB-', 'AB+'
@IsValidPhone()	After normalizePhone (removing spaces and dashes), matches optional + followed by 8 to 15 digits
@IsCalendarDate()	String in format YYYY-MM-DD representing a real calendar date (e.g., '2026-02-31' fails)

DTO Classes and Property Order

Properties in each DTO must be declared in the exact order below so stopAtFirstError: true returns the deterministic message documented in docs/API.md:

DTO	File	Owner	Properties (in checking order)
SignUpDto	auth/dto/sign-up.dto.ts	Eyad	fullName, phone, email (optional), password, bloodType, areaId
SignInDto	auth/dto/sign-in.dto.ts	Eyad	identifier, password
RequestIdParamDto	requests/dto/request-id-param.dto.ts	Adham	id
CreateRequestDto	requests/dto/create-request.dto.ts	Adham	patientReference, bloodType, unitsRequired, areaId, urgency, hospital, note (optional)
UpdateRequestDto	requests/dto/update-request.dto.ts	Adham	unitsRequired (optional), urgency (optional), note (optional)
CancelRequestDto	requests/dto/cancel-request.dto.ts	Adham	cancelReason
RequestFiltersQueryDto	requests/dto/request-filters-query.dto.ts	Adham	status (optional), bloodType (optional), areaId (optional), urgency (optional)

DTO	File	Owner	Properties (in checking order)
RecordDonationDto	donations/dto/record-donation.dto.ts	Adham	donorId, donationDate
UpdateDonorProfileDto	donors/dto/update-donor-profile.dto.ts	Laith	areaId (optional), phone (optional), isAvailable (optional)
PledgeIdParamDto	pledges/dto/pledge-id-param.dto.ts	Laith	id

Validation Pipe (common/validation.pipe.ts — Owner: Eyad)

createValidationPipe() returns a NestJS ValidationPipe configured with:

```
transform: true
whitelist: true
forbidNonWhitelisted: true
stopAtFirstError: true
exceptionFactory → BadRequestException with ONE readable message
string
```

When an unknown property is sent in a body or query, forbidNonWhitelisted: true causes the factory to return 400 Bad Request with { "message": "Unknown field '<property>' is not allowed." }.

8.5 Fixed Service and Controller Method Names

Service	Public methods	Owner
ClockService	now()	Eyad
AppLogger	logRequestCreated(...), logRequestCancelled(...), logRequestFulfilled(...), logDonationRecorded(...), warnDonationRejected(...)	Eyad
AreasService	findAll(), findByIdOrThrow(areaId)	Eyad
AuthService	signUp(dto), signIn(dto), getMe(userId)	Eyad
MatchingService	isBloodCompatible(neededType, donorType), getCompatibleDonorTypes(neededType), getNextAllowedDonationDate(lastDonationDate), isInRestPeriod(lastDonationDate, now), getDonorEligibilityStatus(donor, now), checkDonorEligibilityForReque	Eyad

Service	Public methods	Owner
	st(donor, request, now), getEligibleDonorsForRequest(requestId)	
RequestsService	getPublicSosRequests(), createRequest(coordinatorId, dto), findAllRequests(filters), findRequestById(id), updateRequest(id, dto), cancelRequest(id, dto), getEligibleDonors(id)	Adham
DonationsService	recordDonation(requestId, coordinatorId, dto)	Adham
DonorsService	getMyProfile(userId), updateMyProfile(userId, dto), deactivateMyAccount(userId), getMyEligibleRequests(userId), findAllDonors()	Laith
PledgesService	createPledge(requestId, userId), withdrawPledge(pledgeId, userId)	Laith

Controller methods:

Controller	Methods	Owner
AreasController	getAreas (GET /api/areas)	Eyad
AuthController	signUp (POST /api/auth/sign-up), signIn (POST /api/auth/sign-in), getMe (GET /api/auth/me)	Eyad
RequestsController	getSosRequests (GET /api/requests/sos), createRequest (POST /api/requests), getRequests (GET /api/requests), getRequestById (GET /api/requests/:id), updateRequest (PATCH /api/requests/:id), cancelRequest (PATCH /api/requests/:id/cancel), getEligibleDonors (GET /api/requests/:id/eligible-donors)	Adham
DonationsController	recordDonation (POST /api/requests/:id/donations)	Adham
DonorsController	getMyProfile (GET /api/donors/me),	Laith

Controller	Methods	Owner
	updateMyProfile (PATCH /api/donors/me), deactivateMyAccount (PATCH /api/donors/me/deactivate), getMyEligibleRequests (GET /api/donors/me/eligible-requests), getAllDonors (GET /api/donors)	
PledgesController	createPledge (POST /api/requests/:id/pledges), withdrawPledge (PATCH /api/pledges/:id/withdraw)	Laith

8.6 Module Dependency Graph (Who Calls Whom)

```
AreasController     -> AreasService     -> PrismaService
AuthController      -> AuthService      -> PrismaService,
AreasService, JwtService
RequestsController  -> RequestsService  -> PrismaService,
AreasService, MatchingService, AppLogger
DonationsController -> DonationsService -> PrismaService,
MatchingService, ClockService, AppLogger
DonorsController    -> DonorsService    -> PrismaService,
AreasService, MatchingService, ClockService
PledgesController   -> PledgesService   -> PrismaService,
MatchingService, ClockService
```

Module imports:

```
PrismaModule        (@Global) exports PrismaService
CommonModule        (@Global) exports ClockService, AppLogger
AreasModule         exports AreasService
AuthModule          imports AreasModule, JwtModule; exports
JwtAuthGuard, RolesGuard
MatchingModule      exports MatchingService
RequestsModule      imports AreasModule, MatchingModule; exports
RequestsService
DonationsModule     imports MatchingModule
DonorsModule        imports AreasModule, MatchingModule
PledgesModule       imports MatchingModule
```

There are no circular dependencies between any modules.

8.7 Application Start (api/src/main.ts — Owner: Eyad Ibrahim)

- Global prefix: api
- CORS enabled for http://localhost:5173
- Global validation pipe: createValidationPipe()

- Global exception filter: new AllExceptionsFilter()
- Global guards registered via APP_GUARD in AppModule: JwtAuthGuard first, RolesGuard second (endpoints decorated with @Public() bypass JwtAuthGuard)
- **Swagger setup** at /api/docs with title "Blood Donation Network API", description, version "1.0", .addBearerAuth(), and resource tags (Auth, Areas, Requests, Donations, Donors, Pledges)
- Listens on process.env.PORT || 3000

8.8 api/requests/api.http (Owner: Laith Haj Hosin)

Contains one runnable request for every endpoint and every validation/error scenario in Section 13 and Section 19 of the specification (documented in docs/API.md).

9. Where Each Business Rule Is Enforced

Every business rule is enforced by the backend. The frontend only guides the user and displays backend state and error messages.

Rule ID	Rule Summary	Backend (Enforces & Decides)	Frontend (Guides & Displays)
R-1, R-2	Red-cell blood-type compatibility table	MatchingService.isBloodCompatible & BLOOD_COMPATIBILITY_MAP in common/constants.ts	Never computes compatibility; displays BloodTypeBadge and backend lists
R-3	Single configured rest period (56 days)	REST_PERIOD_DAYS = 56 in common/constants.ts	Displays rest period explanation and backend dates
R-4, R-5	nextAllowedDonationDate = lastDonationDate + 56 days; null if never donated	MatchingService.getNextAllowedDonationDate & isInRestPeriod	Shown on EligibilityBanner, DonorProfilePage, and rejection alerts
R-6	Donor eligible only when active, available, not in rest period, compatible blood type, and same area	MatchingService.checkDonorEligibilityForRequest	Shows only eligible requests on Donor Dashboard and eligible donors on Coordinator screen
R-7	Eligible-donor ranking: exact blood match first, then compatible; oldest lastDonationDate (or never donated) first; then by fullName	MatchingService.getEligibleDonorsForRequest	Renders rows in the exact order returned by the API
R-8	Excluded-donor summary counts compatible donors in same area grouped by restPeriod, unavailable,	MatchingService.getEligibleDonorsForRequest	ExcludedDonorsSummary cards on EligibleDonorsPage

Rule ID	Rule Summary	Backend (Enforces & Decides)	Frontend (Guides & Displays)
	inactive without exposing personal data to public		
R-9, R-14	Request is OPEN, FULFILLED, or CANCELLED; only OPEN requests can be updated, cancelled, pledged to, matched, or receive donations; FULFILLED/CANCELLED cannot be reopened	RequestsService, PledgesService, DonationsService throw 409 Conflict	Update, Cancel, Record Donation, and Pledge buttons only shown when status === 'OPEN'
R-10	unitsStillNeeded = unitsRequired - unitsCollected (never negative)	Calculated in RequestsService & DonorsService response mappers	Displays unitsRequired, unitsCollected, unitsStillNeeded from API
R-11	When unitsCollected reaches unitsRequired, request becomes FULFILLED automatically	DonationsService.recordDonation & RequestsService.updateRequest	Status badge automatically updates to FULFILLED
R-12	unitsRequired cannot be reduced below unitsCollected	RequestsService.updateRequest throws 400 Bad Request	Form input sets min={request.unitsCollected} and displays backend error
R-13	Two OPEN requests with same patientReference and same bloodType are not allowed	RequestsService.createRequest throws 409 Conflict	Shows conflict message inside CreateRequestModal
R-15	Donor can pledge only to an OPEN request they are eligible for	PledgesService.createPledge checks OPEN (409) and MatchingService.checkDonorEligibilityForRequest (400 with exact reason)	Displays backend reason (including nextAllowedDonationDate if in rest period)
R-16	Donor can hold at most one active (PLEDGED) pledge at a time across all requests	PledgesService.createPledge checks existing PLEDGED pledge (409)	Disables pledge buttons or shows backend conflict message when holding an active pledge
R-17	Request cannot have more active (PLEDGED) pledges	PledgesService.createPledge checks activePledgesCount >=	Displays backend conflict message

Rule ID	Rule Summary	Backend (Enforces & Decides)	Frontend (Guides & Displays)
	than unitsStillNeeded	unitsStillNeeded (409)	
R-18	When request becomes FULFILLED or CANCELLED, remaining PLEDGED pledges become RELEASED	DonationsService.recordDonation, RequestsService.updateRequest, RequestsService.cancelRequest inside prisma.$transaction	Pledge badges update to RELEASED
R-19, R-20	Donation recorded only for OPEN request and currently eligible donor; counts as 1 unit	DonationsService.recordDonation verifies OPEN (409) and MatchingService eligibility (400)	Displays error message; nothing changes on failure
R-21	donationDate cannot be in the future	DonationsService.recordDonation checks donationDate <= ClockService.now() (400)	Date picker sets max={today} and shows backend error
R-22	Recording donation, incrementing unitsCollected, completing donor's pledge, updating lastDonationDate, and fulfilling request + releasing other pledges happen in **one database transaction**	prisma.$transaction in DonationsService.recordDonation	Reloads request details and eligible donors after transaction completes
R-23	Public endpoints and SOS board never return donor names, phone numbers, patient references, or notes	RequestsService.getPublicSosRequests selects and returns only public fields	SosBoardPage renders only public fields
R-24	A donor can read and change only their own profile, pledges, and history	DonorsController & PledgesController extract user.id from JWT @CurrentUser()	Donor screens use /donors/me and own pledge IDs

10. What the Backend Calculates

Only the backend calculates or decides:

```
blood-type compatibility (R-1, R-2)
nextAllowedDonationDate and whether a donor is in rest period
(R-3..R-5)
donor eligibility status: ELIGIBLE | REST_PERIOD | UNAVAILABLE |
INACTIVE
```

```
whether a donor is eligible for a specific request and the exact
rejection reason (R-6)
ranked order of eligible donors and excluded-donor counts by reason
(R-7, R-8)
unitsStillNeeded = Math.max(0, unitsRequired - unitsCollected) (R-10)
automatic transition to FULFILLED and pledge release to RELEASED
(R-11, R-18)
```

The frontend must **never**:

- hardcode or evaluate the blood-type compatibility table
- add 56 days to lastDonationDate to decide if a donor is in rest period
- filter or rank eligible donors on the client
- calculate unitsStillNeeded on its own

The only constants in client/src/constants.ts are display lists (BLOOD_TYPES, URGENCY_LEVELS, REQUEST_STATUSES) and input bounds (MIN_UNITS = 1, MAX_UNITS = 10) used strictly to render dropdowns and HTML form attributes.

11. API Service Layer & Auth Token Persistence

```
client/src/services/
├── http.ts           owner: Eyad
├── auth.api.ts       owner: Molham
├── donors.api.ts     owner: Taymaa
├── pledges.api.ts    owner: Taymaa
└── requests.api.ts   owner: Rami
```

File	Exports	Used by
http.ts	request<T>(path, options?), ApiError, getStoredToken(), setStoredToken(token), clearStoredToken()	All *.api.ts service files and AuthContext
auth.api.ts	getAreas(), signUpDonor(payload), signIn(payload), getMe()	AuthContext
donors.api.ts	getMyDonorProfile(), updateMyDonorProfile(payload), deactivateMyAccount(), getMyEligibleRequests(), getAllDonors()	DonorContext, RequestsContext
pledges.api.ts	createPledge(requestId), withdrawPledge(pledgeId)	DonorContext
requests.api.ts	getSosRequests(), getRequests(filters?), getRequestById(id), createRequest(payload), updateRequest(id, payload), cancelRequest(id, payload),	RequestsContext

File	Exports	Used by
	getEligibleDonors(id), recordDonation(requestId, payload)	

How http.ts works

- Reads the JWT token from localStorage under the key bdn.token (wrapped in try/catch).
- If a token exists, automatically attaches Authorization: Bearer <token> to the request headers.
- Parses JSON responses and throws new ApiError(response.status, body.message) when !response.ok.
- Turns network failures (fetch rejection) into new ApiError(0, 'Cannot reach the server. Please check your connection and try again.').

12. TypeScript Types (client/src/types/)

client/src/types/ mirrors docs/API.md 1:1. Eyad creates the initial files in Phase 0, and each frontend owner maintains their respective types file.

```
// auth.types.ts (Owner: Molham)
export type Role = 'COORDINATOR' | 'DONOR';

export type BloodType = 'O-' | 'O+' | 'A-' | 'A+' | 'B-' | 'B+' |
'AB-' | 'AB+';

export interface Area {
  id: string;
  name: string;
}

export interface AuthUser {
  id: string;
  phone: string;
  email: string | null;
  role: Role;
  isActive: boolean;
  donorProfile: {
    id: string;
    fullName: string;
    bloodType: BloodType;
    area: Area;
    isAvailable: boolean;
  } | null;
}

export interface AuthResponse {
  accessToken: string;
  user: AuthUser;
```

```
}

export interface SignUpPayload {
  fullName: string;
  phone: string;
  email?: string;
  password: string;
  bloodType: BloodType;
  areaId: string;
}

export interface SignInPayload {
  identifier: string;
  password: string;
}

// donor.types.ts (Owner: Taymaa)
import { Area, BloodType } from './auth.types';
import { Urgency, RequestStatus } from './request.types';

export type EligibilityStatus = 'ELIGIBLE' | 'REST_PERIOD' |
'UNAVAILABLE' | 'INACTIVE';

export type PledgeStatus = 'PLEDGED' | 'COMPLETED' | 'WITHDRAWN' |
'RELEASED';

export interface DonorActivePledge {
  id: string;
  status: PledgeStatus;
  createdAt: string;
  request: {
    id: string;
    bloodType: BloodType;
    area: Area;
    urgency: Urgency;
    hospital: string;
    unitsRequired: number;
    unitsCollected: number;
    unitsStillNeeded: number;
    status: RequestStatus;
  };
}

export interface DonorDonationHistoryItem {
  id: string;
  donationDate: string;
  units: number;
  request: {
```

```
    id: string;
    bloodType: BloodType;
    hospital: string;
    area: Area;
  };
}

export interface DonorDashboardProfile {
  id: string;
  userId: string;
  fullName: string;
  phone: string;
  email: string | null;
  bloodType: BloodType;
  area: Area;
  isAvailable: boolean;
  isActive: boolean;
  lastDonationDate: string | null;
  nextAllowedDonationDate: string | null;
  eligibilityStatus: EligibilityStatus;
  activePledge: DonorActivePledge | null;
  donationHistory: DonorDonationHistoryItem[];
}

export interface EligibleOpenRequest {
  id: string;
  bloodType: BloodType;
  area: Area;
  urgency: Urgency;
  hospital: string;
  unitsRequired: number;
  unitsCollected: number;
  unitsStillNeeded: number;
  activePledgesCount: number;
  canPledge: boolean;
  createdAt: string;
}

export interface UpdateDonorProfilePayload {
  areaId?: string;
  phone?: string;
  isAvailable?: boolean;
}

export interface CoordinatorDonorItem {
  id: string;
  fullName: string;
  phone: string;
```

```
  email: string | null;
  bloodType: BloodType;
  area: Area;
  isAvailable: boolean;
  isActive: boolean;
  lastDonationDate: string | null;
  nextAllowedDonationDate: string | null;
  eligibilityStatus: EligibilityStatus;
}

// request.types.ts (Owner: Rami)
import { Area, BloodType } from './auth.types';
import { PledgeStatus } from './donor.types';

export type Urgency = 'CRITICAL' | 'HIGH' | 'NORMAL';

export type RequestStatus = 'OPEN' | 'FULFILLED' | 'CANCELLED';

export interface PublicSosRequest {
  id: string;
  bloodType: BloodType;
  area: Area;
  urgency: Urgency;
  hospital: string;
  unitsStillNeeded: number;
  createdAt: string;
}

export interface CoordinatorRequestSummary {
  id: string;
  patientReference: string;
  bloodType: BloodType;
  unitsRequired: number;
  unitsCollected: number;
  unitsStillNeeded: number;
  area: Area;
  urgency: Urgency;
  status: RequestStatus;
  hospital: string;
  note: string | null;
  cancelReason: string | null;
  activePledgesCount: number;
  createdAt: string;
  updatedAt: string;
}

export interface RequestPledgeItem {
  id: string;
```

```
  status: PledgeStatus;
  createdAt: string;
  donor: {
    id: string;
    fullName: string;
    phone: string;
    bloodType: BloodType;
    area: Area;
  };
}

export interface RequestDonationItem {
  id: string;
  donationDate: string;
  units: number;
  createdAt: string;
  donor: {
    id: string;
    fullName: string;
    phone: string;
    bloodType: BloodType;
  };
  recordedBy: {
    id: string;
    phone: string;
    email: string | null;
  };
}

export interface CoordinatorRequestDetails extends
CoordinatorRequestSummary {
  pledges: RequestPledgeItem[];
  donations: RequestDonationItem[];
}

export interface RankedEligibleDonor {
  rank: number;
  id: string;
  fullName: string;
  phone: string;
  bloodType: BloodType;
  isExactBloodMatch: boolean;
  area: Area;
  lastDonationDate: string | null;
  hasActivePledgeForThisRequest: boolean;
}

export interface ExcludedDonorsSummary {
```

```
  restPeriod: number;
  unavailable: number;
  inactive: number;
}

export interface EligibleDonorsResponse {
  requestId: string;
  bloodTypeNeeded: BloodType;
  area: Area;
  eligibleDonors: RankedEligibleDonor[];
  excludedSummary: ExcludedDonorsSummary;
}

export interface RequestFilters {
  status?: RequestStatus;
  bloodType?: BloodType;
  areaId?: string;
  urgency?: Urgency;
}

export interface CreateRequestPayload {
  patientReference: string;
  bloodType: BloodType;
  unitsRequired: number;
  areaId: string;
  urgency: Urgency;
  hospital: string;
  note?: string;
}

export interface UpdateRequestPayload {
  unitsRequired?: number;
  urgency?: Urgency;
  note?: string;
}

export interface CancelRequestPayload {
  cancelReason: string;
}

export interface RecordDonationPayload {
  donorId: string;
  donationDate: string;
}
```

13. Base Colors (client/src/styles/theme.css)

All colors are CSS variables in client/src/styles/theme.css. **Never write a hex or RGB color directly in a page or component CSS file.** In main.tsx, import theme.css first and global.css second.

Variable	Value	Use
--color-primary	#b91c1c	Primary crimson buttons, active links, blood-type badges
--color-primary-dark	#991b1b	Button hover states, emphasis borders
--color-primary-light	#fee2e2	Light crimson tint backgrounds, critical highlights
--color-dark	#1c1917	Navbar, dark hero banner
--color-accent	#fbbf24	Warm highlight on dark backgrounds
--color-critical	#dc2626	CRITICAL urgency badge, urgent alerts
--color-warning	#d97706	HIGH urgency badge, REST_PERIOD banner/badge
--color-info	#2563eb	NORMAL urgency badge, PLEDGED badge
--color-success	#16a34a	ELIGIBLE status, FULFILLED request, COMPLETED pledge
--color-error	#dc2626	Validation errors, rejected operations, INACTIVE badge
--color-bg	#f8fafc	Page background
--color-surface	#ffffff	Cards, modals, tables
--color-border	#e2e8f0	Card and table borders, dividers
--color-text	#0f172a	Primary body text and headings
--color-text-muted	#475569	Secondary labels, timestamps, muted status

For light background tints of semantic colors, use color-mix(in srgb, var(--color-success) 12%, white). Color is never the only signal—every urgency level, blood type, and eligibility state also renders a clear text label.

14. CSS Rules & BEM Prefixes

Plain CSS is global, so class names must never clash across team members.

1. Colors come only from theme.css.
2. Every class starts with its owner prefix using strict BEM notation (prefix__element--modifier).
3. No bare element selectors (button, input, table, h1, a) outside global.css.
4. A page's CSS file is imported only by that page.
5. Never style or override another owner's classes.

Page / Area	Owner	Prefix	Example
Public SOS Board	Molham	sos-board-	.sos-board-page__grid, .sos-board-card__units
Sign In	Molham	sign-in-	.sign-in-page__form
Sign Up	Molham	sign-up-	.sign-up-page__field
Not Found	Molham	not-found-	.not-found-page__title
Shared UI (components/common/)	Molham / Eyad	ui-	.ui-button--primary, .ui-urgency-badge--critical
Donor Dashboard	Taymaa	donor-dashboard-	.donor-dashboard-page__banner--rest
Donor Profile	Taymaa	donor-profile-	.donor-profile-page__toggle
Coordinator Requests	Rami	requests-	.requests-page__filters
Request Details	Rami	request-details-	.request-details-page__counters
Eligible Donors	Rami	eligible-donors-	.eligible-donors-page__excluded-card
Layout & Navbar	Eyad	layout-	.layout-navbar__link

15. Naming Conventions

Data

- JSON fields: camelCase.
- Dates (donation date, next allowed donation date, last donation date): YYYY-MM-DD (example: 2026-12-12).
- Timestamps (createdAt, updatedAt): ISO 8601 string (2026-11-01T09:10:00.000Z).
- Blood types in API & frontend: 'O-' | 'O+' | 'A-' | 'A+' | 'B-' | 'B+' | 'AB-' | 'AB+'.
- Enums in API & frontend: UPPER_SNAKE_CASE (COORDINATOR, DONOR, CRITICAL, HIGH, NORMAL, OPEN, FULFILLED, CANCELLED, PLEDGED, COMPLETED, WITHDRAWN, RELEASED, ELIGIBLE, REST_PERIOD, UNAVAILABLE, INACTIVE).

Backend

- Files: <name>.controller.ts, <name>.service.ts, <name>.module.ts, <name>.guard.ts, <name>.decorator.ts, <name>.util.ts, <name>.types.ts, and <name>.dto.ts inside dto/.
- Classes: PascalCase (DTOs end with Dto). Methods and variables: camelCase. Constants: UPPER_SNAKE_CASE.
- Always use ClockService.now() for current time checks in services. Never call new Date() directly for business-rule comparisons.

Frontend

- Components and pages: PascalCase. Each page has <Name>Page.tsx and <Name>Page.css.

- Contexts: <Feature>Context.tsx, exporting <Feature>Provider and use<Feature>().
- Hooks start with use. Event handlers start with handle. Booleans start with is, has, or can.
- Shared utilities (Owner: **Molham**):

File	Functions	Owner
utils/format.ts	formatDate(dateStr), formatDateTime(isoStr), formatUnits(units), formatPhone(phone)	Molham
utils/date.ts	toDateString(date), getTodayString()	Molham

Git

- Branches: be/<module>-<description> or fe/<page>-<description>.
- Commits: type(scope): message (feat, fix, refactor, style, docs, chore).

16. File Ownership

Every file has **one owner**. Edit only the files you own.

Backend (api/)

Owner	Files
Eyad Ibrahim (Team Lead)	api/prisma/schema.prisma, api/prisma/migrations/, api/prisma/seed.ts, api/src/prisma/, api/src/main.ts, api/src/app.module.ts, api/src/common/ (constants.ts, clock.service.ts, app-logger.service.ts, common.module.ts, normalize.util.ts, validators.ts, validation.pipe.ts, all-exceptions.filter.ts), api/src/auth/ (controllers, services, guards, decorators, DTOs), api/src/areas/, api/src/matching/ (matching.module.ts, matching.service.ts, matching.types.ts)
Adham Albasha	api/src/requests/ (requests.module.ts, requests.controller.ts, requests.service.ts, requests.types.ts, dto/create-request.dto.ts, dto/update-request.dto.ts, dto/cancel-request.dto.ts, dto/request-filters-query.dto.ts, dto/request-id-param.dto.ts), api/src/donations/ (donations.module.ts, donations.controller.ts, donations.service.ts, dto/record-donation.dto.ts)
Laith Haj Hosin	api/src/donors/ (donors.module.ts, donors.controller.ts, donors.service.ts, donors.types.ts, dto/update-donor-profile.dto.ts),

Owner	Files
	api/src/pledges/ (pledges.module.ts, pledges.controller.ts, pledges.service.ts, dto/pledge-id-param.dto.ts), api/requests/api.http

Frontend (client/)

Owner	Files
Eyad Ibrahim (Team Lead)	client/src/App.tsx, client/src/main.tsx, client/src/routes/AppRouter.tsx, client/src/styles/theme.css, client/src/styles/global.css, client/src/components/layout/ (Layout*, Navbar*, ProtectedRoute*), client/src/components/common/PageContainer*, client/src/services/http.ts
Molham Al Kasir	client/src/pages/SosBoard/, client/src/pages/SignIn/, client/src/pages/SignUp/, client/src/pages/NotFound/, client/src/context/AuthContext.tsx, client/src/services/auth.api.ts, client/src/types/auth.types.ts, client/src/components/common/ (Loading*, ErrorMessage*, EmptyState*, Button*, StatusBadge*, UrgencyBadge*, BloodTypeBadge*), client/src/utils/format.ts, client/src/utils/date.ts, client/src/constants.ts
Taymaa Watfeh	client/src/pages/DonorDashboard/, client/src/pages/DonorProfile/, client/src/context/DonorContext.tsx, client/src/services/donors.api.ts, client/src/services/pledges.api.ts, client/src/types/donor.types.ts
Rami	client/src/pages/Requests/, client/src/pages/RequestDetails/, client/src/pages/EligibleDonors/, client/src/context/RequestsContext.tsx, client/src/services/requests.api.ts, client/src/types/request.types.ts

Docs and Repository

Owner	Files
Eyad Ibrahim	docs/, README.md, .github/CODEOWNERS, .github/pull_request_template.md, .editorconfig, .prettierrc, .gitignore, api/package.json, client/package.json

Need a change in someone else's file? Ask the owner or open a GitHub issue. Never edit another member's files directly.

17. Loading, Empty, and Error States

Every screen that loads data must provide:

- a loading view (Loading) while a request is in flight
- an error view (ErrorMessage) displaying the exact backend message and a retry button
- an empty view (EmptyState) whenever a list is empty (such as the SOS board when there are no open requests, eligible requests list, eligible donors list, pledges list, or donation history)
- inline validation and business-rule rejection messages next to forms (never only in the browser console)
- disabled submit/action buttons (isLoading={true}) while a mutation runs to prevent double submissions

18. Architecture Rules

- **Controller → Service → Persistence (Prisma)**: Controllers contain zero business rules and zero PrismaService queries.
- **Centralized Matching**: All blood compatibility, 56-day rest period, and donor eligibility/ranking checks go through Eyad's MatchingService.
- **Single Config for Rest Period**: REST_PERIOD_DAYS = 56 lives only in api/src/common/constants.ts (R-3).
- **Atomic Donation Transaction**: Recording a donation (POST /api/requests/:id/donations) executes all state updates inside a single prisma.$transaction (R-22).
- **Privacy by Default**: Public endpoints (GET /api/requests/sos, GET /api/areas) never return donor names, phone numbers, emails, patient references, or coordinator notes (R-23).
- **Context-Only API Calls**: On the frontend, only AuthContext, DonorContext, and RequestsContext call client/src/services/*.api.ts. Pages and components never call fetch or service functions directly.
- **No Client-Side Rule Duplication**: The frontend never calculates blood compatibility, rest-period end dates, or unitsStillNeeded.
- **CSS Discipline**: No hardcoded colors outside theme.css, and every CSS class uses its owner's BEM prefix.

19. Checks Before Merging

- Prisma schema validates and migrations apply cleanly (npx prisma migrate dev in api/)
- Backend compiles and starts (npm run start:dev in api/) and Swagger loads at http://localhost:3000/api/docs
- Frontend type-checks and builds cleanly (npm run build in client/)
- Feature works end-to-end with the real PostgreSQL database, not only Phase 0 stubs
- Loading, empty, and error states are rendered properly on screen
- Invalid inputs and business-rule rejections show the exact backend message next to the form

- Affected lists and unit counters update immediately after an action without a manual page reload
- Zero console errors and responsive layout on both desktop and mobile viewports
- Only files owned by the PR author were modified